Types of linking

1: static linking /compile time linking

.user defined functiuons are statically linked
.functions from static library are statically linked.

When compiler follows static linking,it will use static library .
If STATIC LIBRARY IS used,then library content will be copied to the execuatble.

main.c

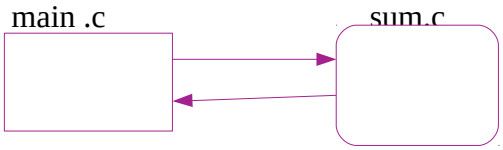
```
int sum(int,int);
int main()

{
   printf("%d",sum(2,3);
}
```

sum.c

```
int sum(int x,int y)
{

    return x+y;
}
```



function main calls sum, sum function is statically linked to main ,if reference of the called function is known during compile time.

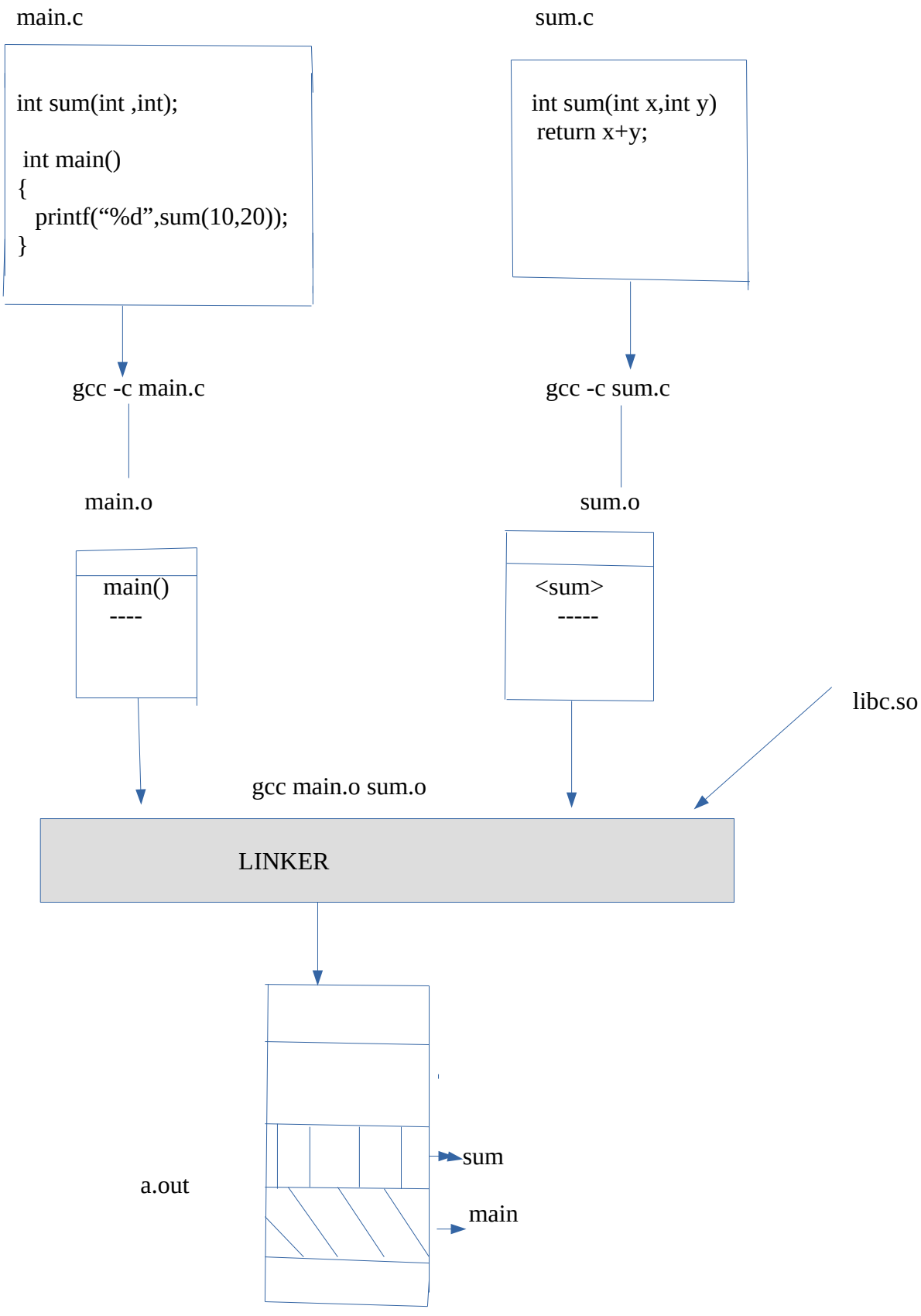
function main calls sum , ,sum function is dynamically linked to main ,if reference is known after the compilet

//
2:dynamic linking

: If refernce of the called function is know after the compilation or known later is known as dynamic linking.

:Dynamic linking is only possible with libary files.

:user defined functions cannot have dynamic linking.

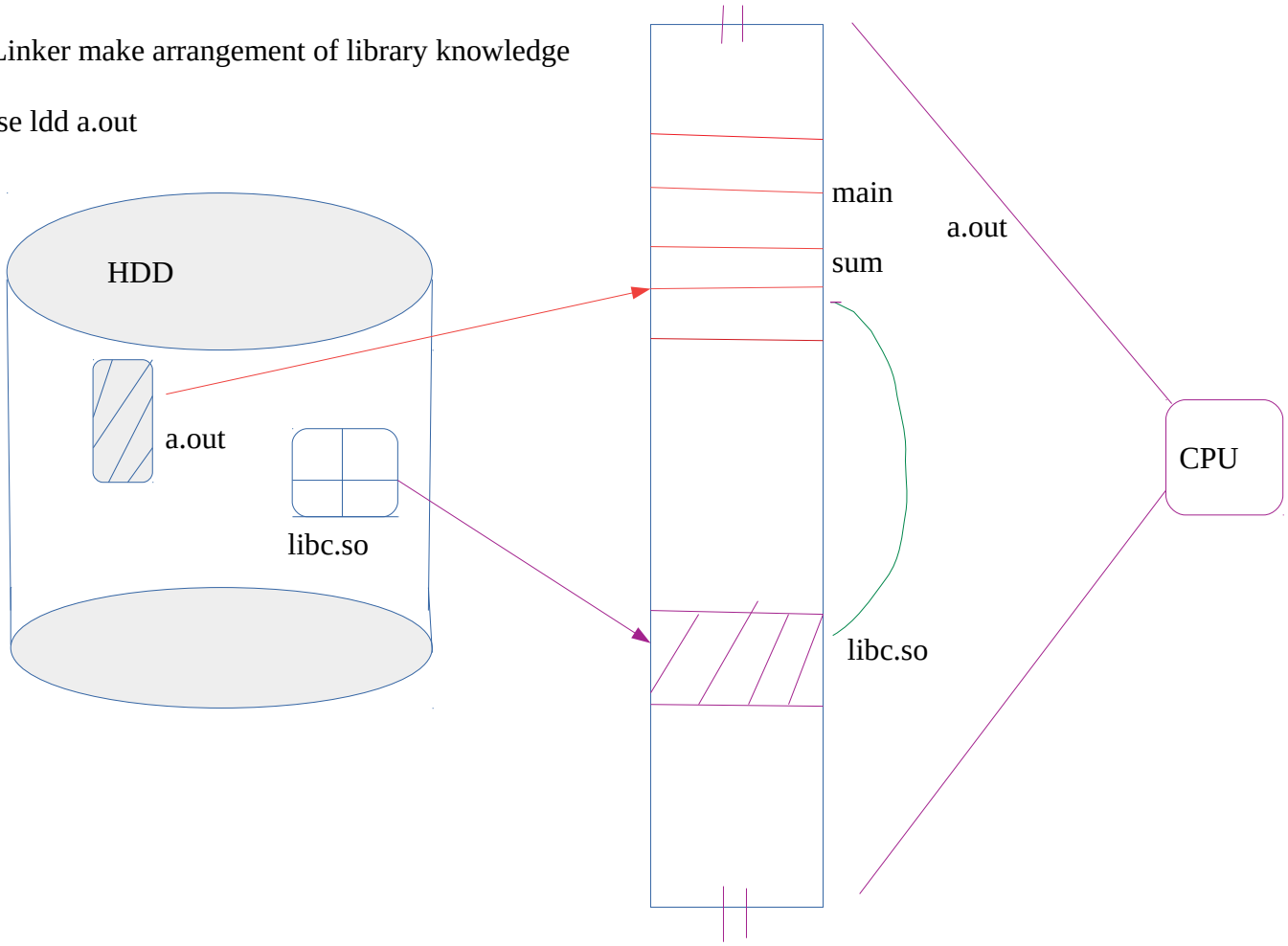

main.c
int sum(int ,int);
int main()
{
printf("%d",sum(10,20));
}
sum.c
int sum(int x,int y)
return x+y;
gcc -c main.c
gcc -c sum.c
main.o
sum.o
main()

<sum>
-----
libc.so
gcc main.o sum.o
LINKER
a.out
sum
main

1:Does main knows the reference of sum?
2:Does main know the reference of printf?
3:No linking error for printf.why?
Use nm a.out
printf --U
sum --- T
main -----T

Linker make arrangement of library knowledge

use ldd a.out



static linking

1:user defined functions are linked at compile time

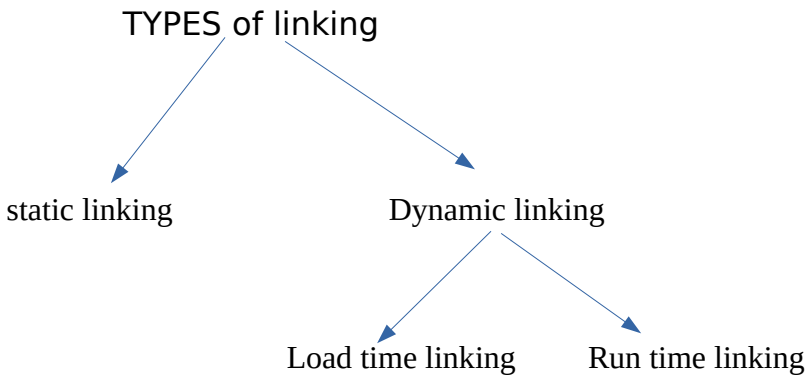
Linker has been providded with STD_C-lib
.Types of library
1.static library --**(.a or .lib)** **(/ find /usr/lib -name "*.a")**

2:**Dynamic Libraries (Shared Libraries) (.so or .dll):**

(// `find /usr/lib -name "*.so*"`)

1. **Static Libraries (.a or .lib):**
    - **File Extension:** `.a` (Unix/Linux), `.lib` (Windows)
    - **Usage:** Compiled directly into the executable at compile time.
    - **Pros:** Faster execution, no runtime dependencies.
    - **Cons:** Larger executable size, each executable has its own copy of the library code.
2. **Dynamic Libraries (Shared Libraries) (.so or .dll):**
    - **File Extension:** `.so` (Unix/Linux), `.dll` (Windows)
    - **Usage:** Linked at compile time or loaded at runtime.
    - **Pros:** Smaller executable size, shared among multiple executables.
    - **Cons:** Potential runtime dependencies, slower startup time.
3. **C Standard Library:**
    - **Examples:** libc (standard C library)
    - **Usage:** Provides fundamental functions for C programming.
    - **Purpose:** `Input/output, string manipulation, memory allocation, etc.`

By using these two lib's , LINKER will perform linking in 2 methods.



. Static linking ----------(static library)

.dynamic linking----------(dynamic lib)

By-default , LINKER follows , LOAD time linking.
while compiling the code , LINKER make some arrangement to link library functions at load time.

What is static linking ?

LINKER will copy library functions/content into exe file.
No dependent file for that application.
For this operation,linker will make use of static library files only
all liibrary and usr defined functions will be link ed at compile time itself.

Libraries come in various types, each serving a specific purpose. Here are some common types of libraries:

How to make LINKER to follow static linking ?????
by using a particular flag , we can make LINKER to follow static linking

flag: - static

1. Lib's are pre-compiled files.
2. we cannot see source code of library file.

we are creating static library with help of AR command by collecting object file (.o)

creation of static lib:

1.create all object files

gcc -c sum.c -o mul.o // creating object file / pre-compiled file

2:By using ar command , collect all object files and store it in one archive file

ar -rcs ./libvector.a sum.o mul.o

Now created static library files

]
3. while compiling , libvector.a need to be supplied

gcc main.o ./libvector.a -o a1.out

executable name (a1.out)

4.a1.out(executable) will be created .

5.check nm a1.out

main -T

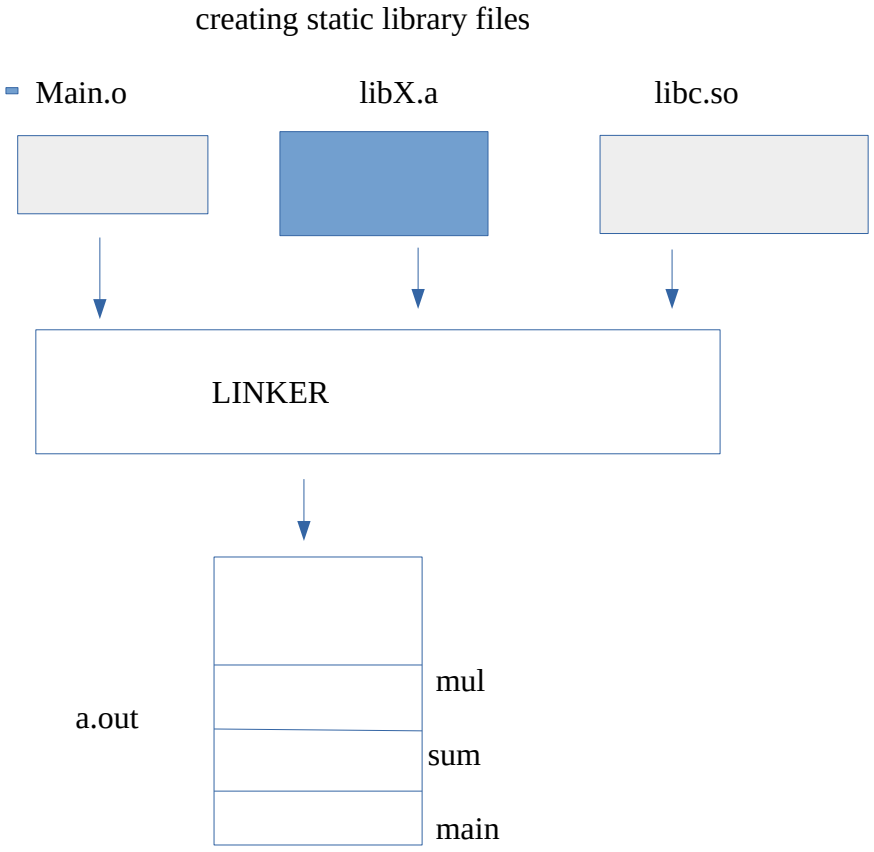
sum -T

MUL -T

PRINTF-U(Undefined)

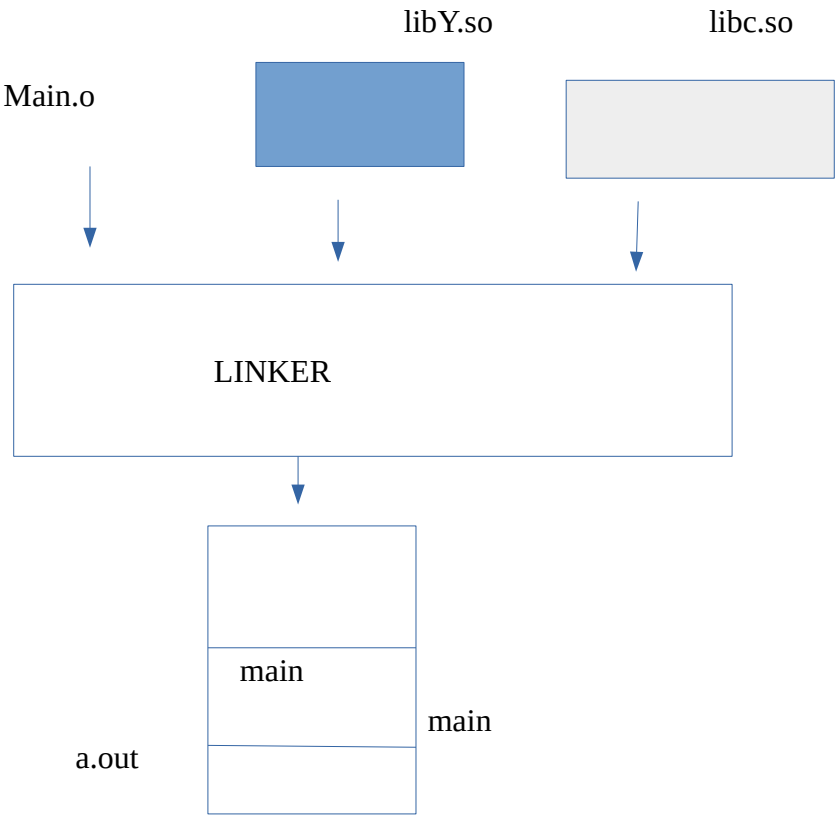
here functions from static library files are statically linked and here predefined function printf will be dynamically loaded at load time.

If we want to make it follow static linking use flag -static
gcc -static main.o ./libvector.a -o a2.out
now all functions,user defined and library functions are copied to the executable.

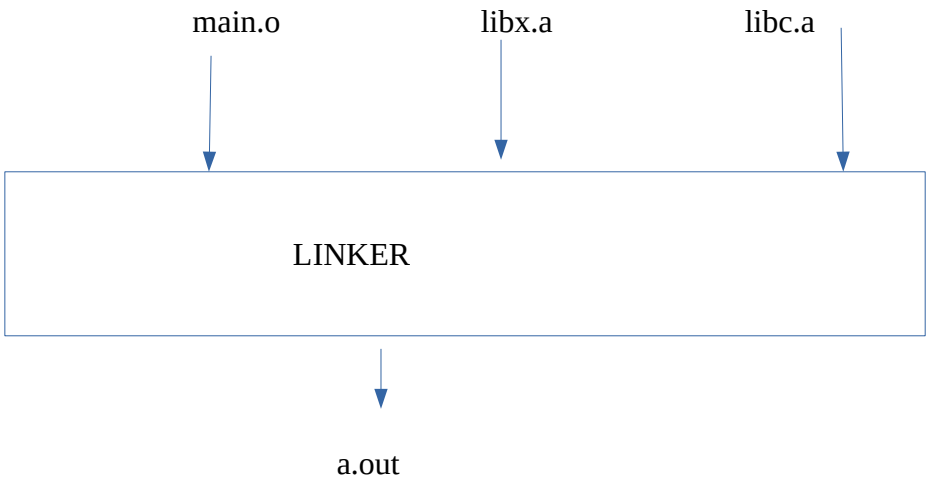


The above example is not 100 % staic linking..

dynamic load time linking



STATIC LINKING-



if we want linker to follow static linking ,use -static flag

gcc -static main.c ./libvector.a -o myprog.exe///(fully static)

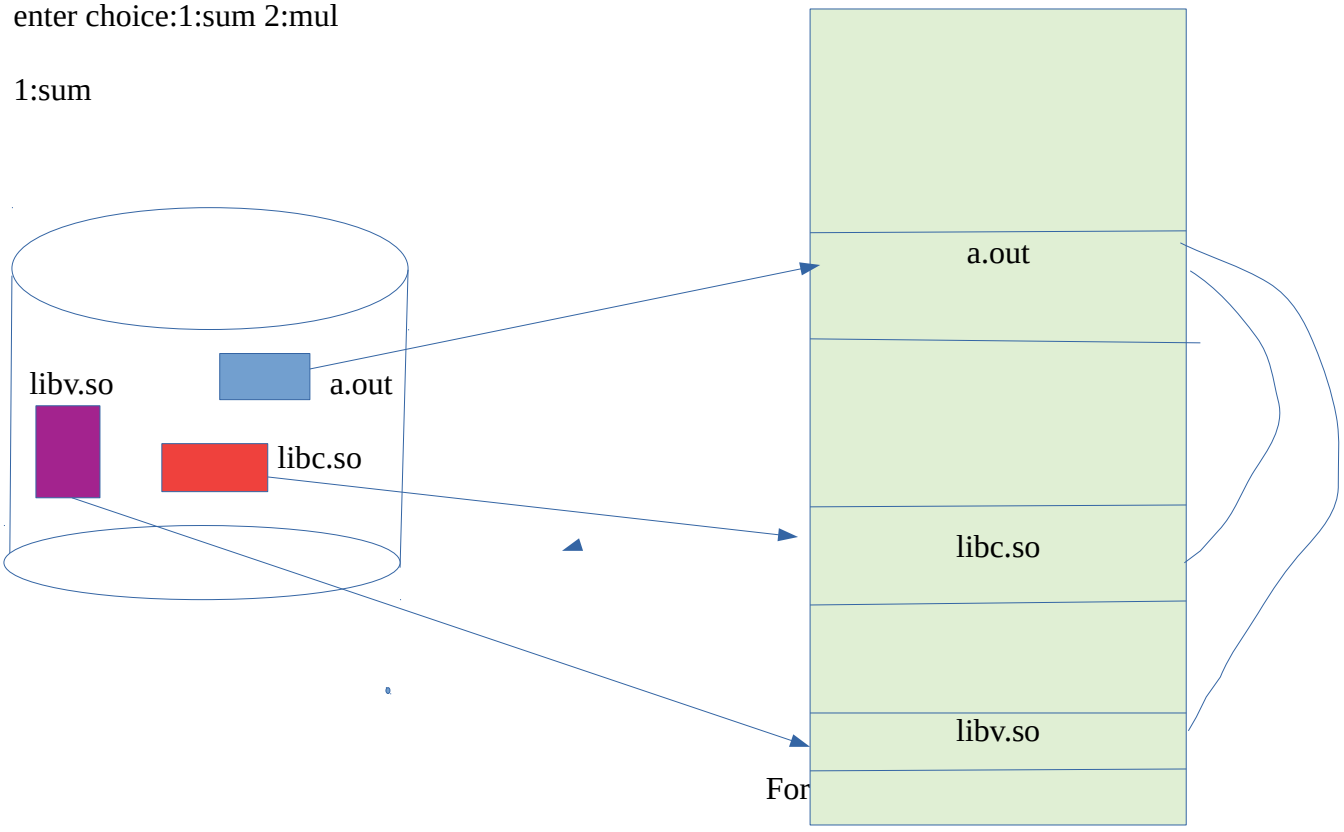
//
1.functions from dynamic library cannot be statically linked.
2.functions from static library cannot be dynamically linked.
//

DYNAMIC LINKING

a..out ,libv.so and libc.so is loaded to the memory

enter choice:1:sum 2:mul

1:sum



Whenever any function is called ,ie, sum or mul is called ,both functions will be loaded to the memory and linked .
Supppose if sum function is called , mul function also will be loaded and linked unwantedly.

So we go for run time linking , whenever needed or on demand certain functions will be loaded to the memory.

Run time linking

--Dynamic library will be used
-Lib function -address will be get resolved at run time(ie,whenevr the specific function is called)
-To maintain run time linking ,we need API(dlopen,dlerror,dlsym,dlclose)

BY using run time linking ,on demand required library can be loaded to the memory.

BY using API, programmer /developer need to write a code to load library into memory.

#include<dlfcn.h> is used

dlopen- open and close a shared object
loads the library file to the memory ,after if successfully loaded then it returns the base address of library ie,where it is present in memory

void *handle///
void pointer can hold any type of address,we dont have libray pointer ,so we have taken void pointer.
Dlsym-it searches for the function

Function pointer/pointer to function

What is function pointer ????
If a pointer able to hold address of a function,then it is known as a function pointer.

if pointer pointing to address which can be executed
1. using ++ / -- operators in invalid. but no syntax errors
2. can not use for DMA

syntax:
1. data type of function pointer should match with return type of function
2. no.of arg's and types of arg's should be match with function pointer arg'

syntax:datatype (*ptr_name) (argtype);

ex: int (*q) (int); //pointer to function

int q(int);

int (*p1) (int ,int)=sum;

```
int (*p2) (int ,int)=mul;
int (*p3)  (int ,int)=div;
```

ARRay of function pointers

```
int (*a[4]) (int ,int);     //typedef int(*FUNPTR)(int ,int); FUNPTR a[4];
```

How to initialize... array of function pointer ??????????????

int arr[3];

Int	int	int

char arr[4]=

Char	char	char

int *p[3]=

Int *	int *	int *

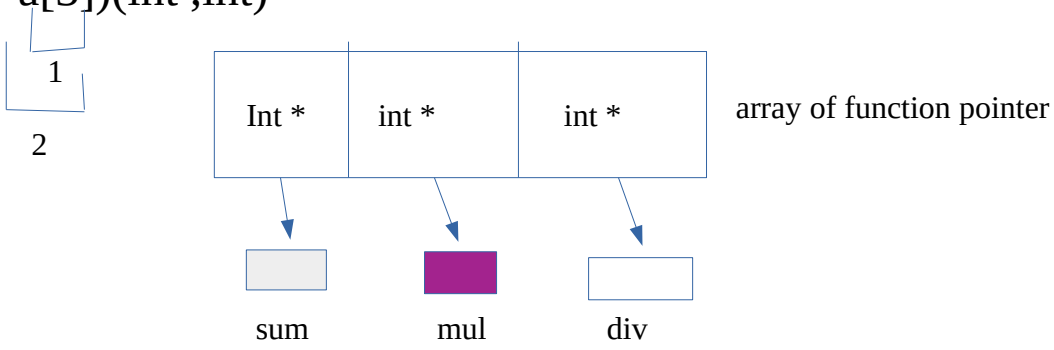
example:int (*a[3])(int ,int)={sum,mul,div);

priority

1.[] -left to right

2.*

int (*a[3])(int ,int)




```
////////////////////////////////////////////////////////////////////////////////
#include<stdio.h>
int add(int x,int y);

int main()
{

 int (*p)(int ,int)=add;//calling indirectlr
int r= p(3,2);

printf("from fnp %d",r);
int res=add(1,2);
printf("%d",res);
```

```
}

int add(int x,int y)
{
  return (x+y);
}


////////////////////////////////////////////////////////////////////////////
#include<stdio.h>

int sum(int x,int y);
int mul(int x,int y);
int div(int x,int y);

int main()

{
	int choice,r;
	int (*p[3])(int,int)={sum,mul,div};/// typedef  int (*ptr)(int,int);
                                                //   ptr p[3]={sum,mul,div};
	int a,b;
	scanf("%d%d",&a,&b);
	printf("enter choice");
	scanf("%d",&choice);
	r=(p[choice-1])(a,b);
	printf("%d",r);
}
int sum(int x,int y)
{
	return (x+y);
}

 int mul(int x,int y)
{
	return (x*y);
}
                int div(int x,int y)
{
	return (x/y);
}



   ////////////////////////////////////////////////////////////////////////////



int  (*a[4]) (int ,int)={sum ,sub,mul,div};

    or can be written as
       typedef int (*FUNPTR) (int ,int);
```

```
        FUNPTR  a[4];


////////////////////////////////////////////////////////////////////////////////

Run time linking
#include<stdio.h>
#include<dlfcn.h>
int main()
{
       int a,b,choice;
       int (*fp)(int,int);    or +///   typedef int (*f) (int ,int);
                              //     f fp;
       void *handle;
    scanf("%d%d",&a,&b);
    printf("1:sum 2:mul");
   printf("enter the choice\n");
   scanf("%d",&choice);
   if(choice ==1)
   {
          handle=dlopen("./libvector.so",RTLD_LAZY);
          if(handle==NULL)
          {
                 printf("%s",dlerror());
                 return 0;
          }
     fp=dlsym(handle,"sum");
     if(fp==NULL)
     {
            printf("%s",dlerror());
            return 0;
     }

     int res=fp(a,b);
     printf("%d",res);


   }

 else if(choice==2)
   {

          handle=dlopen("./libvector.so",RTLD_LAZY);
          if(handle==NULL)
          {
                printf("%s",dlerror());
                return 0;
          }
          fp=dlsym(handle,"mul");
          if(fp==NULL)
          {
```

```
                printf("%s",dlerror());
                return 0;
         }
        int res=fp(a,b);
        printf("%d",res);

   }
   else
         return 0;

}
```